Step 4 — An Analysis Agent That Turns Raw Research Into Organized Findings — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've really tried. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — Organize without losing anything
- Hint 2 — The plan, and the check that matters

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — Organize without losing anything

Analysis turns Research's flat list into 2-4 themes the Writer can build paragraphs from. No tool — organizing is thinking, not looking up.

The trap: "organize these notes" quietly drops a fact that doesn't fit any theme. Say the second rule out loud: **keep every fact and its `[source]` tag**, and put leftovers under "Other".

`run_analysis(notes) -> str` in `agents/analysis_agent.py` — one chain with `analysis.txt`:

- 2 to 4 themes, facts as bullets under each;
- keep every fact and its tag;
- an "Other" theme instead of dropping anything.

Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and the check that matters

```
main():
    topic -> notes = run_research(topic)
    stop if notes == ""
    findings = run_analysis(notes)
    print the findings
    result = write_with_review(findings)
    print_report(result)
```

Test Analysis on Research's **real** output, not on new made-up text — that's what proves the two agents fit together:

```
notes = run_research("electric bikes")
findings = run_analysis(notes)
compare notes.count("[") with findings.count("[")
```

Same number in and out = no fact dropped. Fewer out = fix the prompt before Step 5 hides the problem inside the team.

Full solution: Show me the solution